6. Use a GenAI of your choice to help you create a UML Use Case Diagram for this assignment with the level of detail given in the first UML Use Case Modeling lecture:

a. Identify/discover a use case

1. Sudoku puzzle solver

b. Provide a brief description for the use case

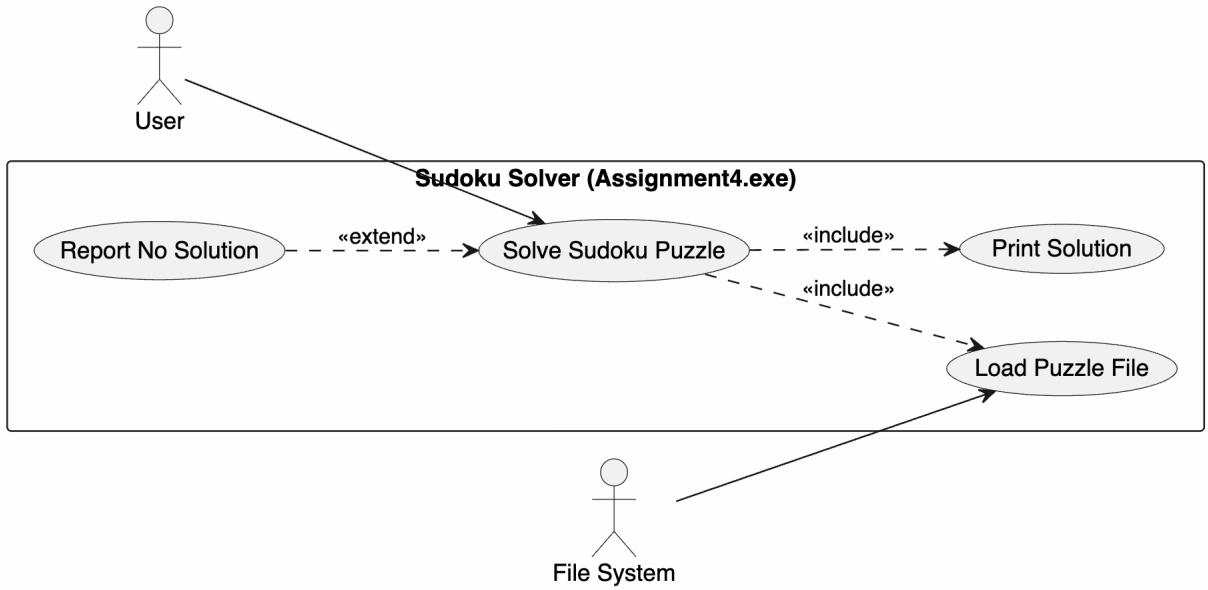
1. The User actor runs the Sudoku solver executable in a directory containing five puzzle files. For each file, the system reads the partially filled 9×9 grid, prints the file name and the puzzle as given, then performs a recursive depth-first search with backtracking to find every valid completion. Each solution found is printed; if the search exhausts all possibilities without completing the grid, the system prints "No solution found." The system repeats this for all five files and terminates.

c. Provide a step-by-step flow (outline) of the events for the use case

1. User launches Assignment4.exe from the command line.
2. System identifies the puzzle files to process (puzzle1.txt … puzzle5.txt).
3. System opens a puzzle file and reads the 9×9 grid, treating _ as an empty cell.
4. System prints the file name and the puzzle as read.
5. System selects an empty cell and computes its candidate digits.
6. System places a candidate and recurses; on dead end, it backtracks and tries the next candidate.
7. When all 81 cells are filled, system records and prints the solution.
8. System continues searching until all branches are exhausted.
9. If no solution was recorded, system prints "No solution found."
10. System repeats steps 3–9 for the remaining files, then exits.

d. Provide a detailed description for the use case

1. User launches Assignment4.exe.
2. System constructs the list of puzzle files to process.
3. System opens the current puzzle file and reads the 9×9 grid into a puzzle object, mapping _ to an empty-cell marker.
4. System prints the puzzle file name.
5. System prints the puzzle exactly as read.
6. System initiates the recursive solve operation on the puzzle object.
7. System selects an empty cell and computes the candidate digits by eliminating values already present in that cell's row, column, and 3×3 block.
8. System places the first candidate in the cell and makes a recursive call to solve the remaining grid.
9. When the recursion reaches a state with all 81 cells filled, system records the grid as a solution and prints it.
10. System backtracks, restores the cell to empty, and tries the next candidate, repeating until every candidate at every level has been tried.
11. If at least one solution was recorded, system moves on; otherwise it prints "No solution found."
12. System repeats steps 3–11 for each remaining puzzle file.
13. System terminates.


User
Sudoku Solver (Assignment4.exe)
Report No Solution
«extend»
Solve Sudoku Puzzle
«include»
Print Solution
«include»
Load Puzzle File
File System